Riwaz Shrestha
Collaborators: No one

1. What are the computation times for different *k* for each of the options? How do the times to compute Fibonacci numbers compare for large *k* between these two options? Are they roughly the same or are they very different? If they are different, what is the multiplicative difference?

    a. For cargo run at k=40, the computation time was 1.856846s. The computation time for cargo run –release was 267.57ms. The two ways to run the program differ in time greatly. One took almost 2 full second while the other took a fraction of the time. The cargo run release for k=40 was close to 7x faster than the cargo run version.

2. Now conduct the following experiment. Replace the array entry type with u8 and adjust any other types accordingly so your program still compiles. Try running the modified code with both cargo run and cargo run --release. Are there any differences in the behavior of the program? If so, what are they?

    a. When the entry type is changed to u8, the program compiles, this results in an overflow error when called using cargo run. When the same program is called with cargo run –release, the program compiles and produces no error. While it runs with no errors, the list produced is not the complete list of Fibonacci numbers. The program outputs part of the Fibonacci sequence but also produces random numbers not a part of the Fibonacci sequence.

3. Explain why the situation described above is not happening, i.e., why the range of integers you use is sufficiently large. This kind of problem is known as *integer overflow*, i.e., you want to explain why integer overflow is not a problem in your code.

    a. Integer overflow is not an error in my code because the number produced by running the code is less than the maximum number that a u32 can hold. The number produced by inputting the maximum number a u8 can hold, 255, is 1065369600. The maximum number that a u32 can hold is 4294967295. U32 is the optimal size because u16 holds less than the number produced by inputting 255 into the program. U16 holds up to 65535 which is less than 1065369600 and that would result in an overflow error.